基于矩阵卷积神经网的多尺度视觉显著物体检测方法

计算机科学与技术 专业

学生 蒋遥 **指导老师** 傅可人

[摘要] 现有基于深度学习和神经网络的显著性物体检测方法通常采用正方形感受野去检测不同的物体。然而，对所有物体都统一采用正方形感受野去进行检测存在一定的缺陷，即对具有较大或较小长宽比的物体往往难以完整地检测出整个物体区域。针对该问题，本文基于矩阵卷积神经网络（Matrix Nets），设计了一种可用于检测显著物体的空洞矩阵卷积模块——AMM（Atrous Matrix Module）,并利用该模块设计并构建了一套用于显著物体检测的卷积神经网络。不同于现有的模型，AMM 使得卷积神经网络具有多长宽比尺度的感受野，能更好地匹配图像中各种物体的形状特性。本文在 6 个广泛采用的公开数据集上进行了实验验证，结果表明，所设计的 AMM 模块以及卷积神经网络比采用正方形感受野的神经网络具有更好的检测效果。

关键词： 显著性物体检测，卷积神经网络，感受野，空洞卷积，多长宽比尺度

Multi-scale visual salient object detection method based on matrix convolution neural network

Computer Science and Techonology

Student Jiang Yao **Adviser** Keren Fu

[Abstract] The existing salient object detection methods based on deep learning and neural network usually use square receptive field to detect different objects. However, there are some defects in using square receptive field to detect all objects. It is difficult to detect the whole object area completely for objects with large or small aspect ratio. To solve this problem, this paper designs an atrous matrix convolution module (AMM) based on matrix convolution neural network (matrix nets) which can be used to detect salient objects, and uses this module to design and construct a set of convolution neural network for salient object detection. Different from the existing models, AMM makes the convolutional neural network have the receptive field of multi-scale aspect ratio, which can better match the shape characteristics of various objects in the image. Experiments on six widely used open datasets show that the AMM module and the convolution neural network have better detection effect than the neural network with square receptive field.

Key words: salient object detection, convolution neural network(CNN), receptive field, atrous convolution, multi-scale aspect ratio

目 录

[摘要] I

[Abstract] II

目 录 III

1 绪论 1

1.1 课题研究的背景及意义 1

1.2 视觉显著性物体检测国内外研究现状 2

1.3 本文组织结构 5

2 开发平台及工具介绍 6

2.1 开发平台 6

2.2 pytorch 机器学习库 6

3 基于矩阵神经网络的显著物体检测方法 7

3.1 矩阵神经网络简介 7

3.2 空洞卷积简介 9

3.3 基于矩阵神经网络的显著性物体检测网络设计 10

3.3.1 AMM 模块 10

3.3.2 基于 AMM 的卷积神经网络 12

4 整体网络实现 15

4.1 网络结构的 pytorch 实现 15

4.1.1 AMM 的 pytorch 实现 15

4.1.2 不具跳跃连接结构的卷积神经网络的 pytorch 实现 18

4.1.3 具有跳跃连接结构的卷积神经网络的 pytorch 实现 18

4.2 网络训练 20

4.3 数据增强 20

5 实验结果 22

5.1 检测结果定量评估 22

5.1.1 验证数据集 22

5.1.2 评价指标 22

5.1.3 实验结果 23

5.2 检测结果定性评估 25

6 总结与展望 27

6.1 总结 27

6.2 展望 27

参考文献 29

致谢 33

1 绪论

1.1 课题研究的背景及意义

视觉作为人的感知之一，是人了解外界、直观感知客观世界的重要途经。视觉信息在人接收的各类信息中占据了绝大部分，在计算机飞速发展的背景下，如何利用计算机对视觉信息进行处理成为一大研究热点，因此，计算机视觉[1] 领域应运而生。计算机视觉是研究如何使计算机像人一样能从视觉信息中“感知”世界的学科，其研究的主要内容是利用计算机代替人去处理各类的视觉信息，并从中提取重要“信息”，其最终目的是使计算机具有类似于人通过视觉信息理解世界的能力[1] 。然而，近年来多媒体信息技术飞速发展，图片和视频质量越来越高，这导致对计算机处理信息的要求也越来越高，此外，在互联网中以图片和视频为载体的信息分享成为常态，图片、视频等信息日益积累，大量急需处理的各种视觉信息给计算机视觉领域带来极大挑战。显著性检测作为计算机视觉的一大研究领域，旨在快速获取图片中最吸引人注意力的兴趣区域（region of interest,ROI）[44] ，将其应用于各类计算机视觉任务，可为计算机提供图像中的关键信息，忽略无关紧要的部分，极大的提升计算机视觉的性能，对显著性检测进行研究，对推动计算机视觉的发展，具有及其重要的意义。

近几年，电子技术飞速发展，各类图片/视频捕捉设备随处可见，同时，随着计算机技术飞速发展，计算机视觉取得重大突破，计算机视觉被广泛应用于学校、公路、银行、超市、医院、机场和地铁等场合，如火车站的自动人脸识别、交通中的非法车辆图像捕捉、医学中的肿瘤识别等。然而对摄像头捕捉到的图像直接进行处理将导致大量计算资源的浪费。因为为了使采集到的物体信息尽量完整，采集设备会同时采集非重要部分，如人脸识别中非人脸部分。在各类计算机视觉任务之前采用显著性检测提取有效信息能有效避免计算资源的浪费。因此对基于矩阵神经网络的多尺度显著性物体检测方法进行研究在节约计算资源方面具有重要意义。

显著性检测在其他计算机视觉任务中同样扮演重要角色，如目标跟踪、图片和视频压缩、自动虚化、基于内容的图像编辑、视频摘要和图像检索等[38] ，进而可广泛用于商业、军事以及研究领域，因此对基于矩阵神经网络的多尺度显著性物体检测方法进行研究并提升其检测性能具有重要意义。

1.2 视觉显著性物体检测国内外研究现状

显著性检测[44] 近年来受到计算机研究者的广泛关注。Itti 等人[2] 于 1998 年运用图像处理算法实现了经典的显著性计算模型，完成了显著性检测领域的开创性工作。显著性检测根据任务地不同，可分为人眼注视预测（eye fixation prediction）和显著性物体检测（salient object detection，SOD）[37] 两大类，人眼注视预测旨在注视人眼在场景中的注视位置，显著性物体检测旨在检测并分割场景中人会自然关注的物体。本文展开的工作属于后者。根据获取特征的方式，现有的显著性物体检测方法可分为传统的显著性物体检测方法和基于深度学习的显著性物体检测方法。

传统显著性物体检测方法。传统的显著性物体检测方法主要利用各种手工设计的特征和先验知识进行显著性物体检测，如显著性物体与其他部分之间颜色、纹理的高对比度、显著性物体的中心先验和边界先验等[39] 。在早期的显著性检测中，受人眼对图像中心-周边差异敏感事实的启发，对比度这一特征被广泛使用[2] 。Ma 等人[3] 通过计算像素与其周围像素间 LUV 颜色向量的欧氏距离来度量像素的显著性，Achanta 等人[4] 提出的显著性检测方法选取像素周围三个不同大小的邻域，利用各领域在 Lab（label color space）色彩空间上均值的欧氏距离来度量像素的显著性。上述方法在计算时仅使用了局部信息，这类基于局部对比度的显著性物体检测方法仅考虑某像素与其领域间的对比度关系，受边界处高对比度的影响其在检测显著性物体的边界时有较好的效果，但是这种方法在检测大尺度显著性物体时会因物体内部的低对比度而丢失显著性物体的内部区域信息。为弥补基于局部对比度方法的严重缺陷，一种基于全局对比度的显著性物体检测方法应运而生，这种方法不同于基于局部对比度的方法，其在全局范围中计算某像素或超像素的对比度并以此衡量该单元的显著性。如，Zhai 等[5] 通过计算图像中全局像素间的颜色距离来衡量某一像素的显著性。此外，Cheng 等[6] 、Achanta 等[7] 同样采用了全局对比度特征。这种基于全局对比度的算法虽然在避免物体内部区域信息的丢失上取得了不错效果，但是又带来了新的不足。基于全局对比度方法以全局范围的对比度作为显著性参考，其得到的显著性区域通常为图像中对比度最大的区域，但是却无法检测出显著性物体的低对比度区域，导致检测出的显著性物体缺失。针对该问题，众多研究者提出了大量基于前景先验和基于背景先验的显著性物体检测方法，该类方法在对比度的基础上引入各种先验的背景或前景知识以帮助获取更加完整的显著性物体，进一步提高显著性检测的性能。基于前景先验的显著性检测方法如[8] [9] 等主要利用颜色对比度以及中心先验等先验知识来评估像素或超像素的显著性。2012 年 Wei 等[9] 在欧洲计算机视觉国际会议（European Conference on Computer Vision）提出了背景先验的概念，之后基于背景先验的显著性检测方法成为显著性检测方向的一大

热潮。这类基于背景先验的显著性物体检测方法[11] [13] 将图像四周区域标注为背景，利用背景与边界相连的先验知识，同时结合对比度等特征对图像中的超像素进行显著性计算。

由于手工设计的特征受到人工特性地限制，这些特征在面对各类变化的图像时不具有普适性，因此这些依赖于各种手工设计的特征与先验的传统显著性物体检测方法不具有很好的鲁棒性。

基于深度学习的显著性物体检测方法。近年来深度学习急速发展，显著性物体检测领域涌现了大量基于深度学习的方法，这些方法极大地推动显著性物体检测的发展。基于深度学习的显著性物体检测又可分为基于经典卷积网络（Convolutional Neural Network, CNN）和基于全卷积网络(Fully Convolutional Networks,FCN)[18] 的模型。

基于 CNN 的显著性物体检测方法。经典卷积神经网络（CNN）通常在卷积层之后连接全连接层用于分类，这种分类器又叫多层感知机（Multi Layer Perceptron,MLP），因此基于经典卷积神经网络的模型又可叫基于多层感知机的模型。基于多层感知机模型按照处理的基本单元可分为基于超像素或块的模型和基于似物性采样（Object Proposal,OP）的模型。基于超像素或块的模型以超像素或块为处理单元，该类模型利用 CNN 对单个单元提取特征并使用 MLP 分类器对其进行显著性预测，基于 OP 的模型以提前生成的物体候选区域或边界框为基本的处理单元，自然而然的编码物体信息[14] 。Zhao 等[15] 提出的多上下文深度学习的显著性物体检测方法（Multi-Context Deep Learning,MCDL）是一种基于超像素的模型，该模型为两路结构，它通过这两条神经网络从中心填充的超像素中分别提取多尺寸的局部和全局上下文信息，再馈送给 MLP 对超像素进行显著性预测，MCDL 模型提高了在背景对比度低且容易造成视觉混淆图片上的性能。不同于 MCDL，MDF[16] （multiscale deep features）和 SSD[17] （shape-based approach for salient object detection）是基于 OP 的模型。MDF 先对图像进行非重叠区域分解，得到 3 级嵌套图，再独立地对 3 级图片进行特征提取，最后馈送给共享的 MLP 进行分类，MDF 的显著性图由 3 级显著性图融合得到。SSD 首先对输入图片进行区域采样，然后使用 CNN 对单个采样区域以及整个图像进行显著性检测，分别获得细化层级的区域显著性图和粗略的全局显著性图，最后将细化层级的区域显著图和粗略的全局显著性图融合并使用 MLP 分类以获取最终的显著性图。

基于经典卷积神经网络的模型相比传统的显著性物体检测模型具有更好的性能和鲁棒性，然而经典卷积神经网络最后的全连接层使得图像的空间信息无法保留。

基于 FCN 的显著性物体检测方法。2015 年，Shelhamer 等[18] 提出的全卷积神经网络（Fully Convolutional Networks,FCN）在语义切割上取得了巨大成功，受此影响，最新的基于深度学习的显著性物体检测模型将流行的分类模型调整为全卷积模型，通过这种端到端的模式，直接输出保留了图像原始空间信息的显著性图，而不是经 MLP 分类器得到的分类得分。基于全卷积神经网络的显著性检测方法发展出了丰富的网络结构，如包含多个网络流的多流网络、具有侧路融合结构的侧融合网络和具有多输出的分支网络[14] ，而不再仅仅是经典 CNN 结构的单流网络。Kuen 等人[19] 提出的循环注意网络（Recurrent Attentional

Networks for Saliency Detection,RACDNN）是典型的单流网络，其主要解决卷积-反卷积这类端到端网络不适用于多尺度物体的问题。RACDNN 在经典 FCN 网络中引入循环神经网络的思想，以此解决网络对多尺度物体检测的细化问题。该方法中，首先以经典的卷积-反卷积生成一个原始粗略的显著图，然后使用 RACDNN 生成一系列显著子图，再用这些子图细化原始显著图。RACDNN 克服了以前基于全卷积神经网络的显著性物体检测方法不适用于多尺度物体的限制，使得端到端的显著性物体检测同样适应多尺度物体。SRM[20]（stagewise refinement model）主要解决下采样操作带来的空间细节和结构损失问题，它通过将显著性图从粗糙的流逐步传递到精细的流中获得精细的显著性图。SRM 首先使用 FCN 网络生成粗糙的预测，然后使用金字塔池化模块（Pyramid Pooling Module,PPM）[20] 逐步融合显著图，来获得更加精细的显著图，该模型在抑制背景噪声上具有显著提升。在眼追踪实验中，通过选择高注意力密度的预分割物体，可对复杂图片中的显著性物体进行明确的标注[21] 。受该事实的启发，Chen 等[21] 提出了 FSN（two-stream fixation-semantic CNNs），FSN 包含并列的注意力流和语义流，FSN 使用两条神经网络提取图片的注意力和语义线索，然后使用基于元素求和进行融合，再将结果放入初始分割模型中来预测显著性。FSN 和 SRM 都具有多个网络流，他们属于多流网络模型。具有短连接的深度监督显著性物体检测方法[22] （Deeply Supervised Salient Object Detection with Short Connections,DSS）是经典的侧融合网络。DSS 在 HED[35] (holistically-nested edge detection)的基础上改进得来，其将深层侧路输出利用短连接的方式融合到浅层侧路输出，利用深层特征中显著性物体的位置信息帮助浅层更好定位，同时利用浅层特征丰富的细节信息帮助细化粗略的显著性图以获取更加精细的显著性图。DSS 最终的显著性图由各层侧路输出融合得到。DHSNet[23] (Deep Hierarchical Saliency Network)首先利用经典的卷积神经网络对输入的整幅图片进行特征提取生成粗略的显著性图，然后重复利用循环卷积层（Recurrent Convolutional Layers,RCL）和下采样层融合局部上下文信息，逐步细化显著性图并最终生成精细的显著性图。DSS 和 DHSNet 均属于侧路融合的网络模型。CPD[24] （Cascaded Partial Decoder）是一个双分支网络，一个分支用于预测 attention，另一个分支用于生成最终的显著性图，attention 预测结果经过整体注意力机制计算后输入另一个分支，用于生成最终结果，CPD 使用额外的卷积模块增大了模型的感受野。

基于 FCN 的模型将全连接层替换为卷积层因而具有保持图像原始空间信息的能力，其与基于经典卷积神经网络的显著性物体检测模型相比大大提高了检测性能。

现有的显著性物体检测方法大多以方形的感受野去检测定位图片中的显著性物体，虽然这种显著性物体检测方法在一些具有挑战的数据集上具有很好表现，但使用这方式对具有不同长宽比的物体进行显著性检测时，往往难以完整的检测及定位出整个显著性物体。

1.3 本文组织结构

论文共包含六章：

第一章，绪论。本章主要介绍课题的研究背景和意义，并对显著性物体检测的国内外研究现状进行了调查分析，最后说明了论文主要内容和组织结构。

第二章，开发平台及工具。介绍了相关研究平台以及所使用的 pytorch 深度学习框架。

第三章，基于矩阵神经网络的显著性物体检测方法。本章对 MatrixsNet 和空洞卷积进行了简单介绍，说明了本文设计的空洞矩阵卷积模块和卷积神经网络。

第四章，整体网络的实现。给出了网络的 pytorch 实现代码，说明了所用的 loss 方法、数据增强以及网络的训练情况。

第五章，实验结果。说明了本文所采用的数据集以及评价指标，通过实验数据分析设计的 AMM 模块和卷积神经网络的有效性。

第六章，总结与展望。本章对本文的研究工作进行总结，并展望显著性物体检测的后续研究。

2 开发平台及工具介绍

2.1 开发平台

本文设计的 AMM 模块及卷积神经网络开发的硬件平台为台式机，使用 i7 8700 CPU，GPU 为单路 1080Ti。计算机操作系统为 windows 10，软件平台为 python 环境，IDE 为 PyCharm，使用 pytorch 深度学习框架。

在 windows 10 上首先安装 python，然后安装 CUDA 以及 CUDA tools，在安装 pytorch 以及相应的库，之后安装 IDE PyCharm。

2.2 pytorch 机器学习库

Torch[36] 是一个开源的机器学习框架，包含大量对于张量（Tensor）的操作、神经网络和基于能量的模型，主要针对大规模的机器学习应用，但受到 Lua 语言的限制 torch 并为被广大开发者推崇。2017 年 Facebook 人工智能研究院团队推出了 torch 的 python 版本，名为 PyTorch[36] 。pytorch 拥有与 torch 相同的底层框架甚至相同的底层语言，不同的是其采用了 python 进行上层封装并提供了大量的 python 接口，使得在 python 中使用 torch 变得极为便利。同为机器学习框架，pytorch 相比 tensorflow 更加简洁。tensorflow 因引入了图、会话、占位符等概念给用户带来大量负担，而 pytorch 则使用张量、变量、模型的三级抽象层次，因而显得更加简洁，同时减小了用户的使用负担。此外，Tensorflow 使用静态的方式构建神经网络的图结构，而 pytorch 则支持动态的网络，用户使用 pytorch 时能以 python 的调试方式对用户构建的网络进行调试，极大的方便了用户对网络的修改和调试，显得更加灵活。除了灵活、简洁的特性，Pytorch 还能对张量计算使用 GPU 加速，同时还具有包含了自动求导系统的深度神经网络[36] 。

Pytorch 的简洁性方便了用户的使用，使得用户更加容易掌握和使用，其灵活性则给用户调试和修改带来极大便利，因此，用户使用 pytorch 机器学习库辅助开发卷积神经网络时能更加专注于卷积神经网络的结构设计，而不需要考虑框架的束缚及消耗时间在底层实现上。

3 基于矩阵神经网络的显著物体检测方法

3.1 矩阵神经网络简介

矩阵神经网络（MatrixNets）是 2019 年 Abdullah Rashwan 等[25] 提出的最新目标检测框架。目标检测旨在检测图像中的各种物体，确定图像中物体的位置以及类别，其包含物体定位和物体分类两个任务。在一个场景中通常包含多个不同尺寸的物体，识别场景中各类尺寸的物体是一个基本而重要的任务[26] 。为解决该问题，各类目标检测方法通常构建特征金字塔网络（Feature Pyramid Network,FPN）[26] 的经典结构，如图：

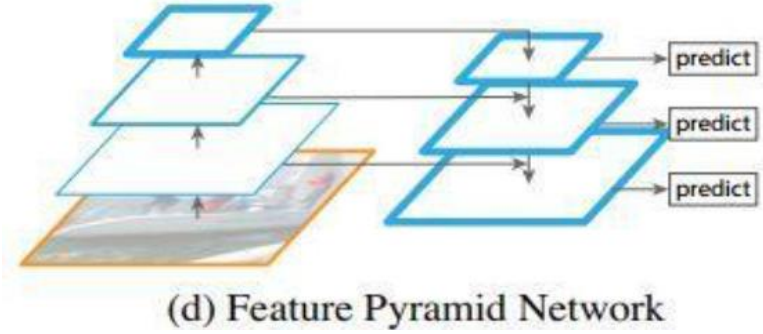


图 1 特征金字塔网络（Feature Pyramid Network）结构[26]

传统金字塔结构通常使用正方形的卷积核对特征图（feature map）进行特征抽取，然后对特征图利用池化技术进行高和宽同比例的下采样，从下到上构建特征金字塔结构，如图 1 所示。FPN 在进行下采样时，同时在宽度和高度方向上使用 1/2 的下采样，通过这种方式得到的上层特征图是下层特征图的 1/2 大小，同时上层特征图的感受野是下层的倍数，由于特征金字塔起始使用方形卷积核提取特征，所以金字塔结构中各特征层所具有的感受野均为正方形。将特征金字塔结构运用于目标检测中，使得场景中不同大小的物体能被具有相应感受野的层捕获，即小物体映射到浅层，大物体映射到深层。虽然特征金字塔结构能够有效解决多尺度物体的检测问题[26] ，但任然存在严重的缺陷。图像中通常存在长高比（apect ratio）不接近 1 的物体，尤其是站立的人或长杆等极端情况的物体，使用特征金字塔网络的结构检测这类物体时，使得物体依据其较大的尺寸被映射到具有相应感受野的层，这个过程中的下采样将导致较小尺寸一方的信息丢失。

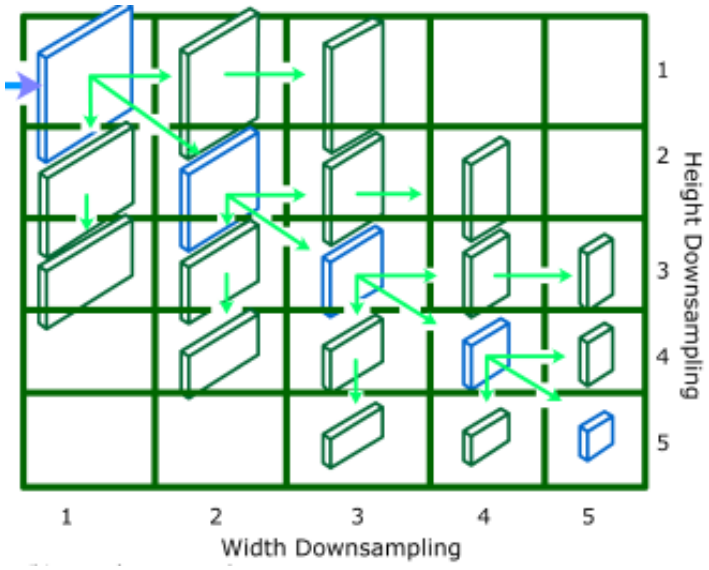


图 2 矩阵网络结构[25]

为解决该问题，Abdullah Rashwan 等[25] 提出了 MatrixNet，如图 2 所示，MatrixNet 在 FPN[26] 的基础上进行改进。MatrixNet 在进行下采样操作时，除了对宽和高同时使用 1/2 的下采样，还增加了分别对宽和高进行 1/2 下采样的操作，同时对宽和高进行下采样得到的特征层作为矩阵的对角层，分别对宽和高进行下采样得到的层则用于构建特征矩阵的左下角和右上角。MatrixNet 使用池化技术对对角线上的特征图进行下采样，对于非对角线的矩阵层则使用 stride 为（2，1）和（1，2）的 3x3 的共享卷积层进行下采样。在 MatrixNet 构建的特征矩阵中，对角线上各层的感受野仍然是正方形，非对角线上的各非对角层的感受野则具多长宽比尺度。在这种结构中，矩阵中每一个矩阵层能处理特定长宽比和特定大小的物体。利用 MatrixNet 进行目标检测时，能有效避免在将特定尺寸和长宽比物体映射到具有相应感受野的层上时小尺寸方向上的信息丢失。MatrixNet 构建非对角层时特征提取使用的卷积核为对角线上共享的卷积核，由于使用共享权值进行特征提取，MatrixNet 在未明显增加参数的前提下获得了长宽比感知能力。

Abdullah Rashwan 等[25] 基于 MatrixNet 提出了 KP-xNet 框架，该框架在目标检测领域取得了巨大成功。KP-xNet 首先使用 MatrixNet 对输入图片进行多长宽比尺度感受野的特征提取，然后对所有矩阵层进行 heatmaps、centers 和 offsets 预测再对各物体进行检测框预测。KP-xNet 会对某一物体以各种感受野预测多个检测框，并使用非极大值抑制（soft non-maximum suppression）选取最合适的目标检测框。

3.2 空洞卷积简介

池化（Pooling）[40] 层是卷积神经网络中常用的结构，其在卷积神经网络中具有极其重要的意义。池化通过将图像分为多个区域并计算每个区域的输出值来模拟人的视觉系统对视觉形象的降维和抽象功能[40]，在卷积神经网络中使用池化能对卷积层提取的特征进行降维以减小数据规模和计算量，此外，还能防止网络过拟合以提高模型的性能。池化对单个区域的输出值具有多种计算方式，最常采用的计算方式为求均值和最大值，分别对应平均池化（Mean Pooling）和最大池化（Max Pooling），此外还有采取其他计算方式的随机池化（Stochastic Pooling）和金字塔池化（Spatial Pyramid Pooling）等[40]。

池化因其具有的特征不变性被早期的非像素级分类任务广泛使用，因为非像素级分类模型更加关注特征存在性及其相对位置，而不关注特征的精确位置。池化操作会同时引起图片尺寸和感受野尺寸的改变，其带来的下采样会导致大量信息丢失，但是语义分割需要进行像素级分类，这要求网络能保留特征的精确信息，因此池化的下采样功能不再适合这类精细任务。此外，池化还具有其他的缺陷，如池化的确定性或不可学习性、过多下采样导致的小物体信息无法重建。为弥补池化的各种缺陷，同时保留其扩大感受野的能力，Fisher Yu 等[27] 提出了空洞卷积（dilated convolution）模型。该模型在卷积的基础上增加了空洞（holes）的概念。空洞的大小以超参数膨胀率率（dilation rate）表示，其具体指的是卷积核各权值在特征图上对应位置间的间隔距离。空洞卷积在进行计算时仅对对应的点进行卷积操作，忽略“空洞”指示的点，如图 3 所示。

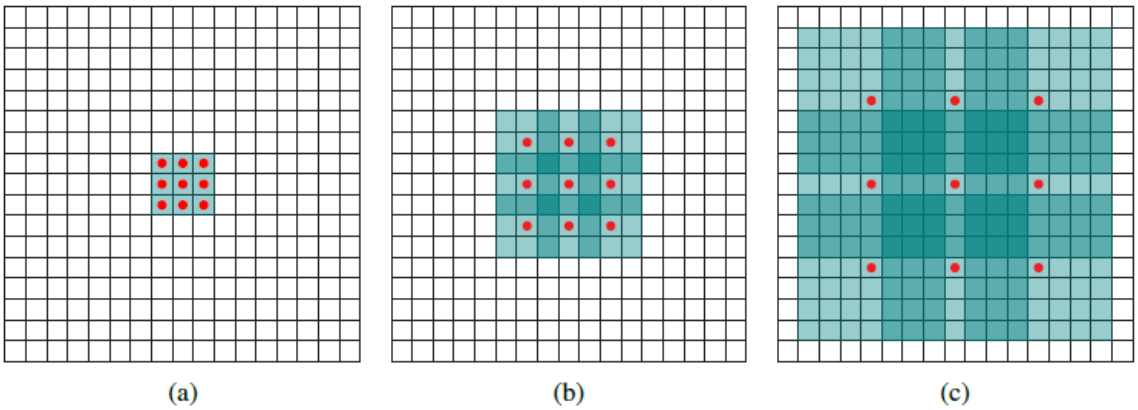


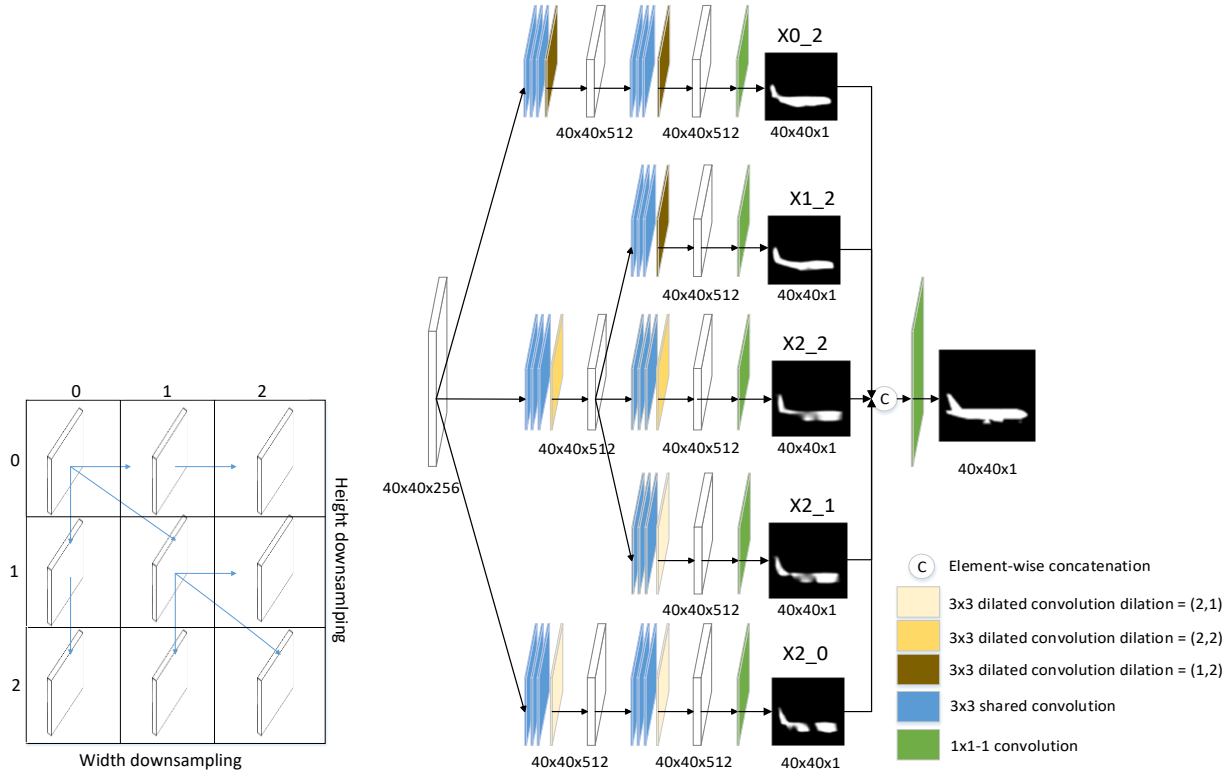
图 3 空洞卷积示意图[27]，a 中 dilation rate 为 0，b 中为 1，c 中为 3

空洞卷积通过在卷积操作中注入空洞，实现了在维持特征图尺寸的前提下扩大网络感受野的目标。空洞卷积解决了池化的下采样导致信息丢失等问题，因其保留了特征的精确信息而适用于像素级分类任务。

3.3 基于矩阵神经网络的显著性物体检测网络设计

受 MatrixNet 启发，本文设计了可用于多尺度显著性物体检测的通用的空洞矩阵卷积模块，并利用该模块设计并构建了一套用于显著性物体检测的神经网络，该模块可广泛使用于显著性物体检测中各类主干网络，如 VGG[26] 、ResNet 等。AMM 模块和所设计的卷积神经网络分别如图 4、图 6 所示。本文设计的基于空洞矩阵卷积模块的显著性物体检测网络以 VGG16[26] 骨干网络为基础，在其上增加了 AMM 模块。本文所设计的卷积神经网络首先利用 VGG16[28] 等骨干网对输入图片进行特征提取，然后将特征图输入 AMM 模型进行多感受野的上下文信息收集，之后将 AMM 模型得到的粗略显著图返还给骨干网，进行细化或者输出。按照常用的 VGG16 分步方法，本文将 VGG16 的卷积部分分为 5 各阶段，X1-X5，分别表示 VGG16 中的 Conv1_1~1_2、Conv2_1~2_2、Conv3_1~3_3、Conv4_1~4_3、Conv5_1~5_3 提取到的特征图。AMM 模块和所设计的卷积神经网络的详细结构将在下章节说明。

3.3.1 AMM 模块



（a）AMM 示意图 (b)AMM 网络结构图

图 4 AMM 模型结构，图中白色方块代表 Feature map,蓝色表示 VGG16 各阶段的卷积层，黄色表示

空洞卷积层，绿色（1x1-1 convolution）表示卷积核为 1x1、输出通道数为 1 的卷积层，Ⓒ表示通道维度的串联，图中仅示意出卷积层，ReLU、BN 等被省略。

AMM 模块示意图如图 4（a）。作为通用的显著性物体检测模块，AMM 的输入为骨干网络的特征图。在显著性物体检测中，虽然存在高塔、站立的人和长杆这类长宽比过大或过小的极端物体，但是其长宽比仍然具有一定限度，因此，使用具有稍大或稍小长宽比尺度感受野的卷积核就能合理地对这类物体进行上下文信息收集。在卷积神经网络中，浅层特征包含的是物体的纹理等细节信息，而高层特征则包含物体位置一类的抽象信息，本文以 VGG16 为例，对 VGG16 的五个阶段进行侧路输出，结果如图 5 所示。

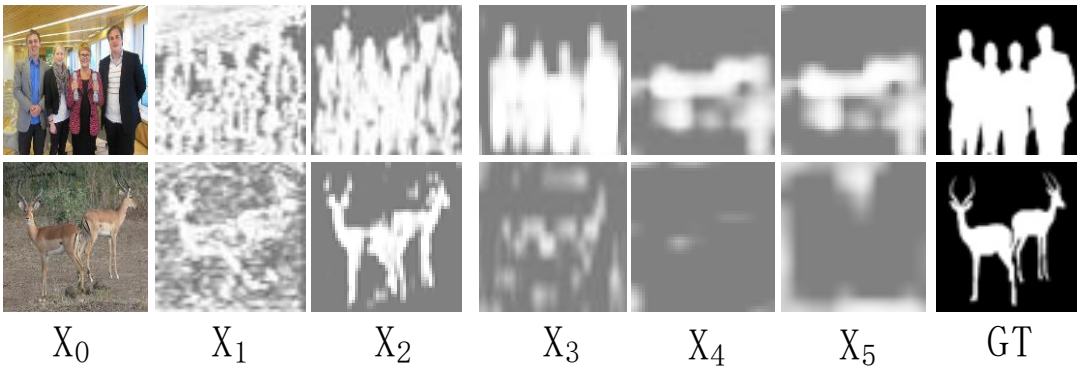


图 5 VGG 网络中各阶段的侧路输出，X1-X5 表示由浅层到深层

综合考虑各类物体长宽比限度以及骨干网中各层包含的信息，本文将 AMM 模块的输入定为 VGG、ResNet 等骨干网的第三阶段输出，即 X3，AMM 既能合理利用较高层的位置信息，又能构建足够检测各类长宽比物体的矩阵层，它在提升网络对各类物体的检测性能和减少计算量之间取得了平衡。

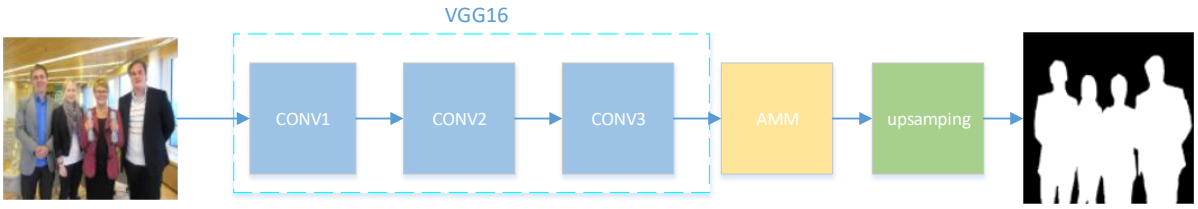
VGG[26] 等骨干网在卷积层中周期性地插入了池化操作，利用池化操作对特征图进行下采样和扩大感受野。MatrixNet 则使用方形卷积核在宽和高方向上进行不同步长的卷积操作，以获取多长宽比尺度的感受野[25] 。然而池化操作带来的下采样将导致图片原始空间信息的丢失，甚至会导致小物体信息无法重建，而在宽和高方向上进行不同步长卷积操作的方法也会导致特征图尺寸的改变，进而引起信息丢失。为了避免获取多长宽比尺度感受野时内部数据结构和空间信息的丢失，AMM 采用更加适合像素级分类任务的空洞卷积（dilated convolution）[27] 代替池化等增大感受野的操作，以获取多长宽比尺度的感受野。

AMM 模块详细的网络结构如图 4（b）所示。图中中心线上的矩阵层代表图 4（a）中的对角线上的层，中心线上侧代表图 4（a）中右上角的层，中心线下侧代表图 4（a）中左下角的层。在空洞矩阵卷积模块的对角线上，如图 4（a），AMM 重复使用 3x3 的卷积和激活函数 ReLU（Rectified Linear Unit）对特征图进行特征提取，再使用膨胀率为（2，2）的空洞卷积在宽和高方向上同时扩大其感受野，获得深层特征图。对于非对角层，如图

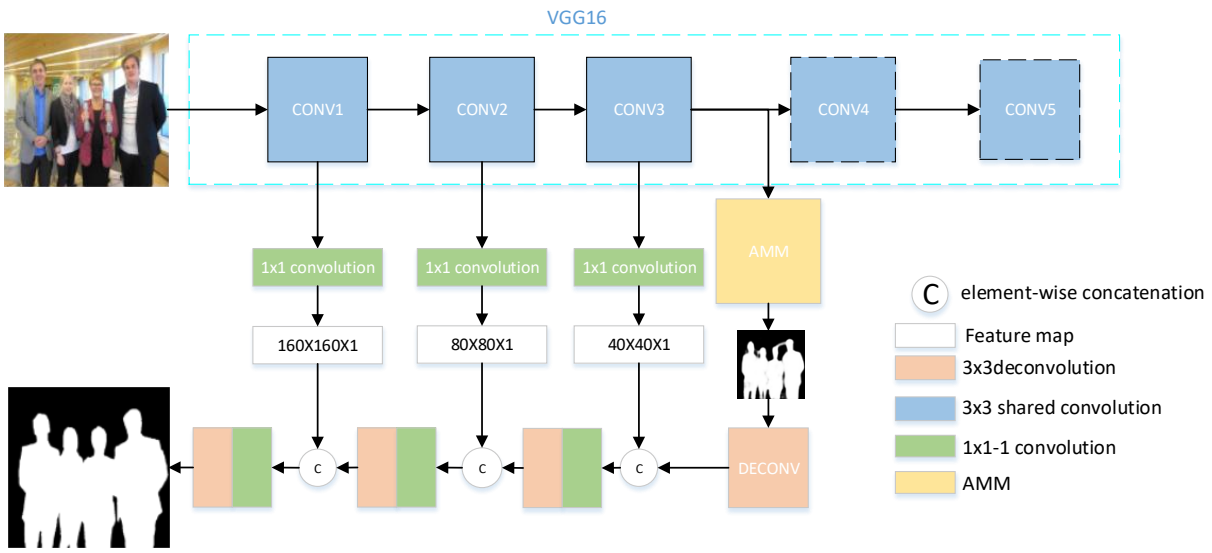
4（a），AMM 重复使用 3x3 的卷积层和激活函数 ReLU 进行特征提取，再用 dilation 为（2，1）和（1，2）的空洞卷积分别在宽和高方向上扩大感受野以获得左下角及右上角的矩阵层。本文在 AMM 的最外层特征图上增加了卷积核为 1x1、输出通道数为 1 的卷积层（1x1-1 convolution），用于对特征图进行特征压缩以获取显著性图，AMM 最外层特征图如图 4 (a) 的最右侧和最下侧所示，具体网络结构如图 4（b）。在 AMM 中，从输入到各矩阵层输出是一条具有特定长宽比尺度感受野的单独卷积神经网络，对角线上的网络具有方形感受野，左侧和上方具有长方形感受野，越偏离对角线，感受野的长宽比越大或越小，如图 4（a）。AMM 对输入的特征图进行计算后将在各矩阵层生成独立的显著性图。由于 AMM 模块基于全卷积网络，本文增加了融合层对各矩阵层的预测结果进行融合以获得最终的显著性图，如图 4（b），本文先对各矩阵层预测结果使用通道维度的串联（element-wise concatenation），再使用卷积核为 1x1、输出通道数为 1 的卷积层获得最终的显著性图。

由于 VGG、ResNet 等骨干网已经具有很强的表达能力，同时在显著性物体检测上具有极高的性能，AMM 共享骨干网中的卷积层进行特征提取，如对于 VGG16，AMM 首先共享其第四阶段卷积层 Conv4_1~4_3 对输入特征图进行特征提取，使用相应的空洞卷积生成多长宽比尺度感受野的矩阵层，再共享 Conv5_1~5_3 进行特征提取使用相应空洞卷积获取 AMM 最外侧矩阵层。在 AMM 中，多长宽比尺度感受野与共享卷积层相结合，使得原骨干网中的方形的卷积核能以合理的感受野收集各类尺寸和长宽比物体的上下文信息。AMM 中所有特征图受空洞卷积特性影响规模不变。

3.3.2 基于 AMM 的卷积神经网络



（a）不具跳跃连接的基于 AMM 的显著性物体检测卷积神经网络，其中蓝色方块表示 VGG16 各阶段的卷积层，黄色方块表示 AMM 模块，绿色方块表示上采样层



(b)具有跳跃连接的基于 AMM 的显著性物体检测卷积神经网络，其中蓝色方块表示 VGG16 各阶段卷积层，绿色方块（1x1-1 convoltion）表示卷积核为 1x1、输出通道数为 1 的卷积层，黄色方块表示 AMM，橙色方块表示反卷积层，Ⓒ表示通道维度的串联，AMM 输出粗略的显著性图

图 6 基于 AMM 的显著性物体检测卷积神经网络结构，图中仅示意出卷积层，ReLU、BN 等被省略。

本文设计的基于 AMM 的卷积神经网络属于基于 FCN 的显著性物体检测方法，其结构如图 6 所示。网络使用 VGG16[26] 骨干网并去掉了其全连接层，此外，网络将骨干网最后两阶段卷积层替换为 AMM 模块。本文设计的基于 AMM 的卷积神经网络包括不具跳跃连接结构的网络如图 6（a）和具有跳跃连接结构的卷积神经网络网络如图 6（b）。在本文中，以 FCN-AMM 表示设计的基于 AMM 的卷积神经网络，SK 表示跳跃连接结构。

不具跳跃连接结构的卷积神经网络（图 6（a））将 VGG 骨干网的第三阶段得到的特征图 X3 输入到 AMM 模型，AMM 共享骨干网的第四、五阶段的卷积层 Conv4_1~4_3 和 Conv5_1~5_3。该网络对 AMM 模型的预测结果进行上采样并直接输出。

具有跳跃连接结构的网络（图 6（b））增加了跳跃连接结构用于细化预测结果。本文采用的跳跃连接结构仿照 U-net。U-net 是 Ronneberger 等人[33] 于 2015 年提出的用于图像分割的全卷积神经网络框架，该网络在 2015 年生物医学成像国际会议（International Symposium on Biomedical Imaging,ISBI）的细胞跟踪挑战赛上取得了巨大成功。U-net 包含收缩路径（contracting path）和扩展路径（expending path）。收缩路径采用的是经典卷积神经网络的架构，其周期性使用 3x3 的卷积、ReLU 以及最大池化。扩展路径则由一系列反卷积层和卷积层构成，其利用反卷积对特征图进行上采样，再将上采样结果与收缩路径中对应的裁剪后的特征图融合并进行卷积操作以获得该阶段的特征图，各阶段特征图重复该操作直到获得最终的显著性图。本文设计的具有跳跃连接结构的网络仿照 U-net 网络结构。该网络首先对骨干网中各阶段特征图增加了 1x1 的卷积层进行特征压缩，得到各阶段特征

图的侧路输出。之后，该网络将 AMM 输出的粗略显著性图使用反卷积层（deconvolution）上采样到上一层特征图大小，并使用通道维度的串联（element-wise concatenation）将侧路输出与粗略的显著性图融合，再使用 1x1 的卷积层进行特征压缩以获得新的显著性图，对新的显著性图重复该操作实现对显著性图的逐步细化，直到获得最终显著性图。融合过程可表述为：

$$new_X_i = W_i * cat(X_i, deconv(new_X_{i+1})) + b_i$$

其中 i∈{1,2,3,4}，X_i 表示由下到上第 i 层侧路输出,特别地 X_4 表示 AMM 输出的粗略显著性图。

4 整体网络实现

4.1 网络结构的 pytorch 实现

AMM 模块以及基于 AMM 的卷积神经网络借助 pytorch 深度学习框架实现，其具体实现代码如下：

4.1.1 AMM 的 pytorch 实现

```
class AMM (nn.Module):
    def __init__(self, params, n_class):
        super().__init__()
        self.n_class = n_class
        self.relu = nn.ReLU(inplace=True)
        self.params = params
        self.convx35 = nn.Conv2d(512, n_class, kernel_size=1)
        self.convx45 = nn.Conv2d(512, n_class, kernel_size=1)
        self.convx53 = nn.Conv2d(512, n_class, kernel_size=1)
        self.convx54 = nn.Conv2d(512, n_class, kernel_size=1)
        self.convx5 = nn.Conv2d(512, n_class, kernel_size=1)
        self.right34 = nn.Conv2d(512,512,kernel_size= 3,padding=(1,2),dilation=(1,2))
        self.right35 = nn.Conv2d(512, 512, kernel_size=3, padding=(1, 2), dilation=(1,
2))
        self.right45 = nn.Conv2d(512, 512, kernel_size=3, padding=(1, 2), dilation=(1,
2))
        self.down34 = nn.Conv2d(512, 512, kernel_size=3, padding=(2,1), dilation=(2,1))
        self.down35 = nn.Conv2d(512, 512, kernel_size=3, padding=(2,1), dilation=(2,1))
        self.down45 = nn.Conv2d(512, 512, kernel_size=3, padding=(2,1), dilation=(2,1))
        self.diagonally34 = nn.Conv2d(512, 512, kernel_size=3, padding=2, dilation=2)
        self.diagonally45 = nn.Conv2d(512, 512, kernel_size=3, padding=2, dilation=2)
        self.classifier = nn.Conv2d(5, n_class, kernel_size=1)
    def forward(self, x):
```

```
#****************************************生成空洞矩阵卷积网络（AMM）
        diagonallyx3_4 = self.relu(F.conv2d(x3, weight= self.params[14],
bias=self.params[15], stride=1, padding=1))
        diagonallyx3_4 = self.relu(F.conv2d(diagonallyx3_4 , weight= self.params[16],
bias= self.params[17], stride=1, padding=1))
        diagonallyx3_4 = self.relu(F.conv2d(diagonallyx3_4 , weight= self.params[18],
bias= self.params[19], stride=1, padding=1))
        diagonallyx4 = self.diagonally34(diagonallyx3_4)
        diagonallyx4_5 = self.relu(F.conv2d(diagonallyx4 , weight= self.params[20],
bias= self.params[21], stride=1, padding=1))
        diagonallyx4_5 = self.relu(F.conv2d(diagonallyx4_5 , weight= self.params[22],
bias= self.params[23], stride=1, padding=1))
        diagonallyx4_5 = self.relu(F.conv2d(diagonallyx4_5 , weight= self.params[24],
bias= self.params[25], stride=1, padding=1))
        diagonallyx5 = self.diagonally45(diagonallyx4_5)

        x3_4 = self.relu(F.conv2d(x3, weight= self.params[14], bias= self.params[15],
stride=1, padding=1))
        x3_4 = self.relu(F.conv2d(x3_4, weight= self.params[16], bias= self.params[17],
stride=1, padding=1))
        x3_4 = self.relu(F.conv2d(x3_4, weight= self.params[18], bias= self.params[19],
stride=1, padding=1))
        x3_4 = self.right34(x3_4)
        x3_5 = self.relu(F.conv2d(x3_4, weight= self.params[20], bias= self.params[21],
stride=1, padding=1))
        x3_5 = self.relu(F.conv2d(x3_5, weight= self.params[22], bias= self.params[23],
stride=1, padding=1))
        x3_5 = self.relu(F.conv2d(x3_5, weight= self.params[24], bias= self.params[25],
stride=1, padding=1))
        x3_5 = self.right35(x3_5)

        x33_4 = self.relu(F.conv2d(x3, weight= self.params[14], bias= self.params[15],
stride=1, padding=1))
        x33_4 = self.relu(F.conv2d(x33_4, weight= self.params[16], bias=
self.params[17], stride=1, padding=1))
```

```
x33_4 = self.relu(F.conv2d(x33_4, weight= self.params[18], bias= self.params[19], stride=1, padding=1))
x33_4 = self.down34(x33_4)
x33_5 = self.relu(F.conv2d(x33_4, weight= self.params[20], bias= self.params[21], stride=1, padding=1))
x33_5 = self.relu(F.conv2d(x33_5, weight= self.params[22], bias= self.params[23], stride=1, padding=1))
x33_5 = self.relu(F.conv2d(x33_5, weight= self.params[24], bias= self.params[25], stride=1, padding=1))
x33_5 = self.down35(x33_5)

x4_5 = self.relu(F.conv2d(diagonallyx4, weight= self.params[20], bias= self.params[21], stride=1, padding=1))
x4_5 = self.relu(F.conv2d(x4_5, weight= self.params[22], bias= self.params[23], stride=1, padding=1))
x4_5 = self.relu(F.conv2d(x4_5, weight= self.params[24], bias= self.params[25], stride=1, padding=1))
x4_5 = self.right45(x4_5)

x44_5 = self.relu(F.conv2d(diagonallyx4, weight= self.params[20], bias= self.params[21], stride=1, padding=1))
x44_5 = self.relu(F.conv2d(x44_5, weight= self.params[22], bias= self.params[23], stride=1, padding=1))
x44_5 = self.relu(F.conv2d(x44_5, weight= self.params[24], bias= self.params[25], stride=1, padding=1))
x44_5 = self.down45(x44_5)
#*****************************************************************获取结果
x3_5 = self.convx35(x3_5)
x33_5 = self.convx53(x33_5)
x4_5 = self.convx45(x4_5)
x44_5 = self.convx54(x44_5)
diagonallyx5 = self.convx5(diagonallyx5)
score = torch.cat((diagonallyx5, x4_5, x44_5, x3_5, x33_5), dim=1)
score = self.classifier(score)
score = F.interpolate(score, size=(320, 320), mode='bilinear', align_corners=True)
```

```
        return score
```

4.1.2 不具跳跃连接结构的卷积神经网络的 pytorch 实现

```
class FCN-AMM-noSK(nn.Module):
    def __init__(self, pretrained_net, n_class):
        super().__init__()
        self.n_class = n_class
        self.pretrained_net = pretrained_net
        self.AMM = AMM(self.pretrain_net.named_parameters,self.n_class)
    def forward(self, x):
        output = self.pretrained_net(x)
        x5 = output['x5']
        x4 = output['x4']
        x3 = output['x3']
        x2 = output['x2']
        x1 = output['x1']
        score = self.AMM(x3)
    return score
```

4.1.3 具有跳跃连接结构的卷积神经网络的 pytorch 实现

```
class FCN-AMM-withSK(nn.Module):
    def __init__(self, pretrained_net, n_class):
        super().__init__()
        self.n_class = n_class
        self.pretrained_net = pretrained_net
        self.AMM = AMM(self.pretrain_net.named_parameters,self.n_class)
        self.relu = nn.ReLU(inplace=True)
        self.deconv1 = nn.ConvTranspose2d(512, 512, kernel_size=3, stride=1,
padding=1, dilation=1, output_padding=0)
        self.bn1      = nn.BatchNorm2d(512)
        self.deconv2 = nn.ConvTranspose2d(512, n_class, kernel_size=3, stride=1,
padding=1, dilation=1, output_padding=0)
        self.bn2      = nn.BatchNorm2d(n_class)
```

```
        self.deconv3 = nn.ConvTranspose2d(1, n_class, kernel_size=3, stride=2,
padding=1, dilation=1, output_padding=1)
        self.bn3     = nn.BatchNorm2d(n_class)
        self.deconv4 = nn.ConvTranspose2d(1, n_class, kernel_size=3, stride=2,
padding=1, dilation=1, output_padding=1)
        self.bn4     = nn.BatchNorm2d(n_class)
        self.deconv5 = nn.ConvTranspose2d(1, n_class, kernel_size=3, stride=2,
padding=1, dilation=1, output_padding=1)
        self.bn5     = nn.BatchNorm2d(n_class)
        self.sidex1= nn.Conv2d(64, n_class, kernel_size=1)
        self.sidex2= nn.Conv2d(128, n_class, kernel_size=1)
        self.sidex3= nn.Conv2d(256, n_class, kernel_size=1)
        self.A1 = nn.Conv2d(2, n_class, kernel_size=1)
        self.A2 = nn.Conv2d(2, n_class, kernel_size=1)
        self.A3 = nn.Conv2d(2, n_class, kernel_size=1)
        self.A1 = nn.Conv2d(2, n_class, kernel_size=1)
        self.classifier = nn.Conv2d(1, n_class, kernel_size=1)
    def forward(self, x):
        output = self.pretrained_net(x)
        x5 = output['x5']
        x4 = output['x4']
        x3 = output['x3']
        x2 = output['x2']
        x1 = output['x1']
        score = self.AMM(x3)
        x1 = self.sidex1(x1)
        x2 = self.sidex2(x2)
        x3 = self.sidex3(x3)

        score = self.bn1(self.relu(self.deconv1(score)))
        score = self.bn2(self.relu(self.deconv2(score)))
        score = self.relu(self.A3(torch.cat((score, x3), dim=1)))
        score = self.bn3(self.relu(self.deconv3(score)))
        score = self.relu(self.A2(torch.cat((score, x2), dim=1)))
        score = self.bn4(self.relu(self.deconv4(score)))
```

```
        score = self.relu(self.A1(torch.cat((score, x1), dim=1)))
        score = self.bn5(self.relu(self.deconv5(score)))
        score = self.classifier(score)
        score = F.interpolate(score, size=(320, 320), mode='bilinear', align_corners=True)
    return score
```

4.2 网络训练

上述所有的网络在 1080Ti 上训练，训练集选取 DUTS[29] 数据集，使用预训练的 VGG16[28] 初始化共享的卷积层，其他卷积层使用 pytorch 的默认初始化。训练时 batch size 设置为 10，优化方法选取随机梯度下降法（Stochastic Gradient Descent,SGD），学习率设为 1e-2，动量设为 0.7。此外，本文使用二项交叉熵（Binary Cross Entropy）对预测结果进行监督。多个样本的二项交叉熵损失表示如下：

$$loss(X,Y) = -1/N \sum_{j=0}^{N} \sum_{i} (y^j{}_i \log(x^j{}_i) + (1 - y^j{}_i) \log(1 - x^j{}_i))$$

其中，X 表示显著性图，Y 表示 Ground truth，N 表示 batch size，x^j_i，y^j_i 分别表示第 j 个图片第 i 个像素的显著性值和 Ground truth 值。二分类交叉熵损失函数的调用代码如下：

```
LossFunction = troch.nn.BCELoss()
loss = LossFunction(output,ground_truth)
```

4.3 数据增强

在网络训练中，为保证网络的泛化能力，本文采取了数据增强的手段，包括翻转、随机裁剪。在使用随机裁剪时，为避免生成过小的物体，本文在图片的宽和高方向上分别随机选取连续 80%部分。随机裁剪的方式如下：

```
def CROP(A,B):
    h = A.size[0]
    w = A.size[1]
    x = random.randint(0,20)
    y = random.randint(0,20)
    x = x*0.01*h
    y = y*0.01*w
    A = A.crop([x,y,h,w])
    B = B.crop([x,y,h,w])
```

return A,B

此外，因 AMM 模型具有长宽比感知能力，为了使得获取的上下文信息维持原状，训练的数据使用时保持图片原始长宽比，即在保证原始长宽比的前提下投入网络训练。

5 实验结果

5.1 检测结果定量评估

5.1.1 验证数据集

为验证模型的有效性，本文选取了 DUTS[29] 、DUT-OMRON[30] 、PASCAL-S[31] 、ECSSD[32] 、HKU-IS[32] 、MSRA-B[43] 六个常用的基准数据集作为验证集。

5.1.2 评价指标

本文选取了四个普遍使用的评价指标对基于矩阵网络的显著性物体检测方法进行评估。这四个评价指标分别是精确率-召回率（Precission-Recall,PR）曲线[46]，F-measure[46]，S-measure[42] 和平均绝对误差（mean absolute error ,MAE）[41]，F-measure 和 S-measure 得分越高，MAE 得分越低，表示检测结果越好。

PR[6] 曲线中 P 代表精确率，R 代表召回率。其公式表述为：

$$Precision(T) = \frac{|\mathrm{M(T)} \cap \mathrm{Gt}|}{|M(T)|} \qquad Recall(T) = \frac{|\mathrm{M(T)} \cap \mathrm{Gt}|}{|\mathrm{Gt}|}$$

其中 M（T）是通过用阈值 T 直接对显著图 S 进行阈值化而获得的二值掩码，|·|是图内掩码的总面积。通过改变 T 可以得到精确的 PR 曲线。

F-measure[6] 公式表述为：

$$F - measure = \frac{(1+\beta^2)\mathrm{Precision} \cdot \mathrm{Recall}}{\beta^2 \cdot \mathrm{Precision} + \mathrm{Recall}}$$

β 是用于平衡 Precison 和 Recall 的权值，此处 β^2 设为 0.3。

S-measure[42] 公式表述为：

$$S - measure = \alpha * S_0 + (1-\alpha) * S_r$$

其中 α 是对象感知结构相似性 S_o 和区域感知结构相似性 S_r 之间的平衡参数,此处将 α 设为 0.5。

MAE[41] 定义如下：

$$MAE = \frac{1}{W * H}\sum_{x=1}^{W}\sum_{y=1}^{H}|S_{map}(x,y) - Gt(x,y)|$$

$S_{map}(x,y)$和 Gt(x,y)分别表示在像素（x,y）处的显著值和 Ground truth 值，W 和 H 表示图片的宽和高。

5.1.3 实验结果

本文选取了经典的 FCN 网络作为对比网络，包括含有跳跃连接结构的和不含跳跃连接结构的 FCN 网络，FCN 最后两阶段的卷积部分类似于 AMM 的对角线部分，不同地是，FCN 使用池化操作扩大感受野，而 AMM 使用空洞卷积扩大感受野。FCN-AMM 在 FCN 的基础上，将最后两个阶段的卷积部分替换为 AMM 模块。为了使比较更加公平，本文将所有模型在 DUTS 的训练集上进行重新训练。本文分别对上述四种模型在六个数据集上测量了其 F-measure，S-measure、MAE 以及 PR 曲线，分别如表 1、表 2、表 3，图 7 所示。

表 1 四种模型在六个数据集上的 F-measure 得分比较。SK 表示跳跃连接结构，FCN-AMM 表示所设计的卷积神经网络，FCN 表示基于 VGG16 的全卷积神经网络

Methods\Datasets	DUTS	DUT-OMRON	PASCAL-S	ECSSD	HKU-IS	MSRA-B
FCN(without SK)	0.6925	0.6209	0.7562	0.8427	0.8108	0.8938
FCN-AMM(without SK)	0.7448	0.6531	0.7905	0.8748	0.8630	0.9316
FCN(with SK)	0.7721	0.6813	0.7982	0.8931	0.8868	0.9400
FCN-AMM(with SK)	0.7837	0.6961	0.8084	0.8957	0.8909	0.9388

表 2 四种模型在六个数据集上的 S-measure 得分比较。SK 表示跳跃连接结构，FCN-AMM 表示所设计的卷积神经网络，FCN 表示基于 VGG16 的全卷积神经网络

Methods\Datasets	DUTS	DUT-OMRON	PASCAL-S	ECSSD	HKU-IS	MSRA-B
FCN(without SK)	0.7705	0.7252	0.7838	0.8418	0.8260	0.8995
FCN-AMM(without SK)	0.8029	0.7428	0.8054	0.8666	0.8665	0.9285
FCN(with SK)	0.8210	0.7639	0.8107	0.8818	0.8844	0.9362
FCN-AMM(with SK)	0.8302	0.7769	0.8209	0.8840	0.8891	0.9347

表 3 四种模型在六个数据集上的 MAE 得分比较。SK 表示跳跃连接结构，FCN-AMM 表示所设计的卷积神经网络，FCN 表示基于 VGG16 的全卷积神经网络

Methods\Datasets	DUTS	DUT-OMRON	PASCAL-S	ECSSD	HKU-IS	MSRA-B
FCN(without SK)	0.0777	0.0930	0.1030	0.0795	0.0713	0.0485
FCN-AMM(without SK)	0.0657	0.0884	0.0922	0.0646	0.0509	0.0320
FCN(with SK)	0.0594	0.0792	0.0879	0.0568	0.0432	0.0280

FCN-AMM(with SK)	0.0590	0.0778	0.0869	0.0583	0.0434	0.0299

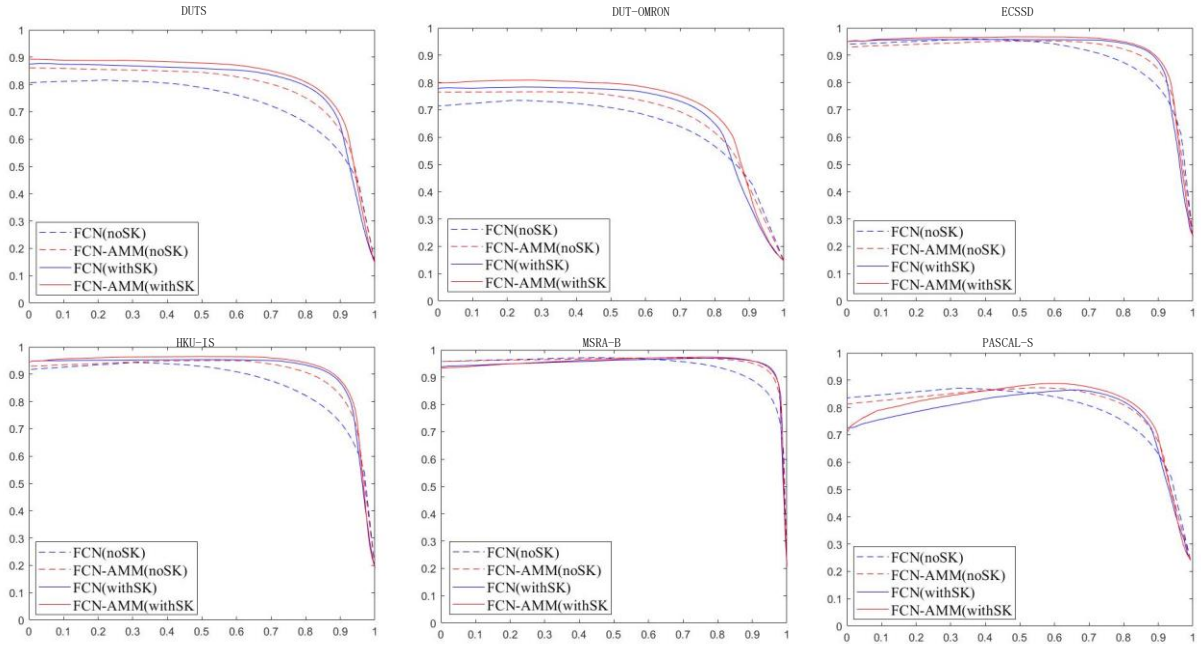


图 7 四种模型在六个数据集上的 PR 曲线，虚线表示不具有跳跃连接结构的网络，实线表示具有跳跃连接结构的网络，蓝色表示基于 VGG16 的 FCN 网络，红色表示设计的卷积神经网络

从表 1、表 2 和表 3 可知，不使用跳跃连接结构的 FCN-AMM 与 FCN 相比较，基于 AMM 的全卷积神经网络在 F-measure、S-measure 以及 MAE 上都有显著的提升。在 DUTS 数据集上，AMM 模块使得网络的预测结果在 F-measure 上提升了 5.23%，在 S-measure 上提升了 3.24%，在 MAE 上下降了 0.012，在其他五个数据集上，F-measure 平均提升了 3.77%，在 HKU-IS 上效果尤为明显，提升了 5.22%；S-measure 平均提升了 2.886%，其中在 HKU-IS 上提升最大；MAE 平均下降了 0.013。

使用跳跃连接结构的 FCN-AMM 与 FCN 相比较，AMM 模块使得网络的预测结果在 DUTS 数据集上的 F-measure 提升了 1.16%，S-measure 提升了 0.94%，MAE 没有明显提升。在其他五个数据集上，F-measure 除了在 MSRA-B 数据集上有略微下降，在其他数据集均有约 1%的提升，S-measure 在 DUTS、DUT-OMRON、PASCAL-S 上提升了约 1%，在 ECSSD、HKU-IS 上提升了约 0.5%，在 MSRA-B 上有轻微下降，而 MAE 在 DUT-OMRON 和 PASCAL-S 上由轻微得分提升，在其他数据集上没有明显效果。

根据四种模型在六个数据集上的 PR 曲线，如图 7，无论是否使用跳跃连接结构，使用 AMM 模块的网络在各数据集上都有更好的 PR 曲线，AMM 对不具跳跃连接结构的网络具有更加明显地效果。

在不使用跳跃连接结构的模型对比中，AMM 使得网络预测结果在 F-measure、S-measure 和 MAE 上的得分显著提高，证明了本文设计的 AMM 模块在显著性物体定位和检测方面具有极大帮助。在使用跳跃连接结构的模型对比中，除 MSRA-B 数据集，基于 AMM 的卷积神经网络在各数据集上的 F-measure、S-measure 得分均优于 FCN 网络，在 DUTS、DUT-OMRON、PASCAL-S 上的 MAE 得分优于 FCN 网络，进一步证明 AMM 模块在显著性物体检测中的有效性，AMM 模型带来的性能提升是跳跃连接结构所不能替代的。

5.2 检测结果定性评估

为了确认 AMM 模型的有效性，本文还进行了视觉上的比较，如图 8、图 9 所示。本文对 AMM 模型中各矩阵层进行了输出并与 FCN 的输出进行比较（如图 8），以证明 AMM 模型带来的性能提升来源于多长宽比尺度感受野，而不是来自多层融合。此外，本文还将基于 AMM 的卷积神经网络的预测结果与 FCN 相比较（如图 9）以证明使用 AMM 在显著性物体检测中的有效性。

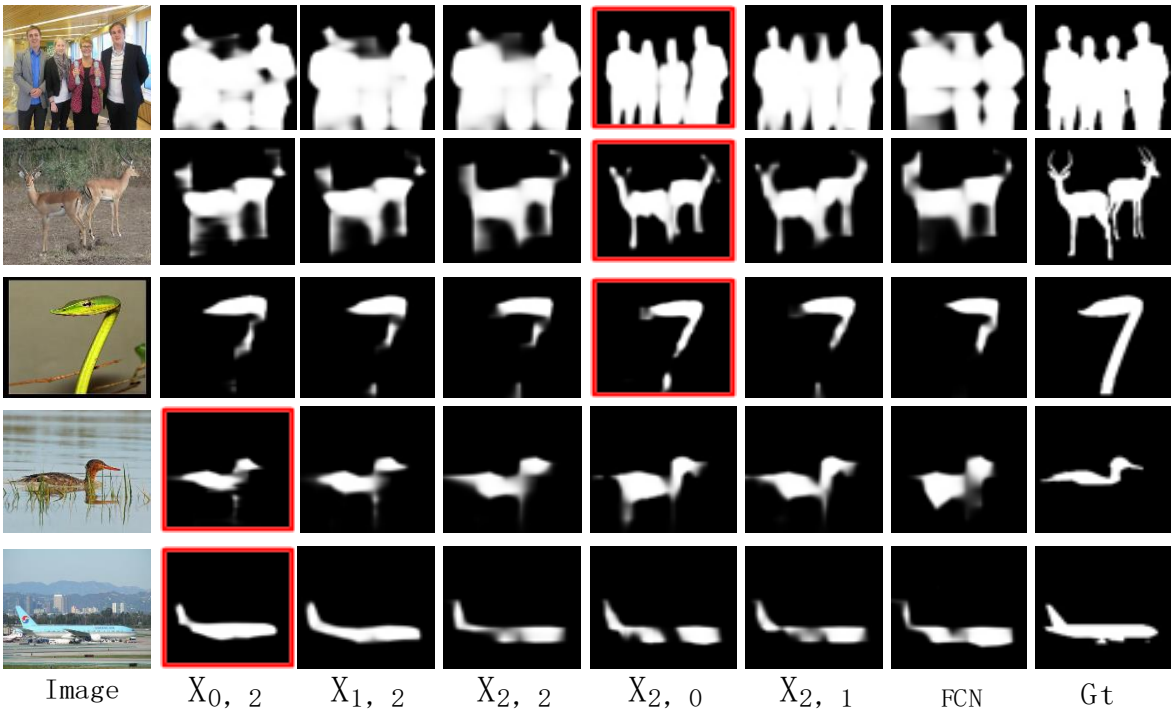


图 8 AMM 各矩阵层输出与 FCN 输出。第一列为输入图片，最后一列为 Ground truth，第七列为 FCN 输出结果,中间各列为对应的矩阵层输出结果，红框标记的显著性图表示视觉上效果最好的预测结果

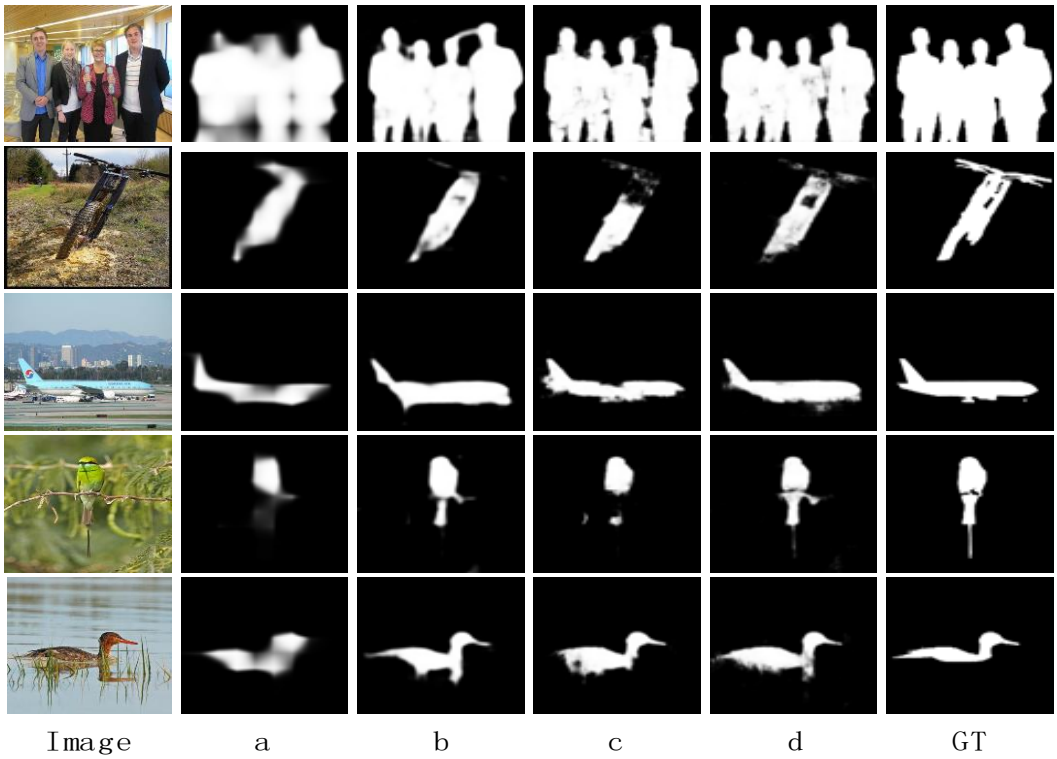


图 9 基于 AMM 的显著性物体检测模型与 FCN 模型预测结果。第一行为输入图片，最后一列为 GT（ground truth），a、b 为不具跳跃连接结构的 FCN 网络和 FCN-AMM 网络的输出，c、d 为具有跳跃连接结构的 FCN 网络和 FCN-AMM 网络的输出

从图 8 中可知，AMM 中具有不同感受野比例的各矩阵层预测结果不一。对具有各种长宽比的显著性物体，AMM 各层输出中检测效果更好的显著性图如图 8 红框标记，这些结果明显优于 AMM 其他层和 FCN 网络预测结果，检测出的物体具有更加精细和更加完整的结构，这证明，对于具有多长宽比尺度的显著性物体，使用骨干网中方形的卷积核以合适的比例和尺寸的感受野去检测物体能更加精细、完整地检测出显著性物体，AMM 模块对显著性物体检测的性能提升来源于多长宽比尺度感受野，而非来自多层融合。

从图 9 中明显可知，基于 AMM 的卷积神经网络的预测结果比 FCN 预测结果有更加完整和精细的结构，这进一步证明具有多长宽比尺度感受野的 AMM 模块在多长宽比尺度显著性物体的检测中的高效性。

6 总结与展望

6.1 总结

显著性物体检测是当今计算机视觉领域的一大热点，它在物体检测和识别、图像切割、图像和视频压缩[38]、视觉跟踪等计算机视觉任务中具有广泛的应用前景，在市场中更具有极大的经济价值。

本文对显著性物体检测方法进行了研究，并基于矩阵神经网络设计了可用于显著性物体检测的空洞矩阵卷积模块，此外，本文利用该模块设计并构建了一套用于显著物体检测的卷积神经网络。本文的研究成果如下：

（1）本文基于矩阵神经网络，设计了可用于显著性物体检测的空洞矩阵卷积模块（AMM）。AMM 使用空洞卷积分别对特征图进行宽和高方向的感受野扩大，得到具有多长宽比尺度感受野的特征矩阵，并用共享权值的方式进行特征提取。AMM 使得骨干网的方形卷积能以合适比例和尺寸的感受野对多长宽比尺度显著性物体进行特征提取。

（2）利用 AMM 模块设计并构建了一套用于显著物体检测的卷积神经网络。

（3）本文在 6 个广泛采用的数据集上进行实验，证明了 AMM 在显著性物体地定位上的高效性和设计基于 AMM 的卷积神经网络的有效性。从实验经过中可知，在对具有各种长宽比尺度的物体进行显著性物体检测时，AMM 在不具有跳跃连接结构的 FCN 网络中有助于检测出物体更加完整的结构，而在具有跳跃连接结构的 FCN 网络中，AMM 也有助于更加精细、完整地检测出显著性物体。

6.2 展望

在本论文的研究工作中，作者有阅读了大量国内外关于显著性物体检测的文献，对显著性物体检测领域和基于全卷积神经网络的显著性物体检测方法有了系统的了解和深刻的认识。在大量的阅读和实验中，作者发现了一些显著性检测领域现存的问题。主要有：

（1）基于全卷积神经网络的显著性物体检测方法在显著性图的精细结构上仍有待提高。对于不具有高对比度边界的显著性物体，在使用基于全卷积神经网络的显著性物体检测方法以及本文设计的卷积神经网络对其进行检测时，往往导致边界信息丢失甚至区

域丢失。此外，使用同类方法对具有低对比度区域的显著性物体进行检测时，得到的显著性图往往不具备精细的结构。

（2）在对场景复杂、显著性物体易于混淆的图片进行显著性物体检测时，基于全卷积神经网络的显著性物体检测方法得到的显著性图通常包含背景噪声。

因此，未来显著性物体检测可在抑制背景噪声和提高显著图中显著性物体边界和整体精细程度上进行进一步地研究。

参考文献

[1] Prince S J D. Computer vision: models, learning, and inference[M]. Cambridge University Press, 2012:15-20.

[2] Itti, Laurent, Christof Koch and Ernst Niebur. A Model of Saliency-Based Visual Attention for Rapid Scene Analysis. IEEE Transactions on pattern analysis and machine intelligence, 1998, 20(11): 1254-1259.

[3] Ma, Yu-Fei and HongJiang Zhang. Contrast-based image attention analysis by using fuzzy growing. Proceedings of the eleventh ACM international conference on Multimedia. 2003: 374-381.

[4] Achanta, Radhakrishna, Francisco J. Estrada, Patricia Wils and Sabine Süsstrunk. Salient Region Detection and Segmentation. International conference on computer vision systems. Springer, Berlin, Heidelberg, 2008: 66-75.

[5] Zhai, Yun and Mubarak Shah. Visual attention detection in video sequences using spatiotemporal cues. Proceedings of the 14th ACM international conference on Multimedia. 2006: 815-824.

[6] Cheng, Ming-Ming, Niloy Jyoti Mitra, Xiaolei Huang, Philip H. S. Torr and Shi-Min Hu.Global Contrast Based Salient Region Detection. IEEE Transactions on Pattern Analysis and Machine Intelligence, 2014, 37(3): 569-582.

[7] Achanta, Radhakrishna, Sheila S. Hemami, Francisco J. Estrada and Sabine Süsstrunk. Frequency-tuned salient region detection. 2009 IEEE conference on computer vision and pattern recognition. IEEE, 2009: 1597-1604.

[8] Jiang H, Wang J, Yuan Z, et al. Automatic salient object segmentation based on context and shape prior[C].BMVC. 2011, 6(7): 9.

[9] Yan, Qiong, Li Xu, Jianping Shi and Jiaya Jia. Hierarchical Saliency Detection. Proceedings of the IEEE conference on computer vision and pattern recognition. 2013: 1155-1162.

[10] Wei Y, Wen F, Zhu W, et al. Geodesic saliency using background priors[C].European conference on computer vision. Springer, Berlin, Heidelberg, 2012: 29-42.

[11] Tong N, Lu H, Zhang Y, et al. Salient object detection via global and local cues[J]. Pattern Recognition, 2015, 48(10): 3258-3267.

[12] Jiang P, Vasconcelos N, Peng J. Generic promotion of diffusion-based salient object detection[C].Proceedings of the IEEE International Conference on Computer Vision. 2015: 217-225.

[13] Jiang B, Zhang L, Lu H, et al. Saliency detection via absorbing markov chain[C].Proceedings of the IEEE international conference on computer vision. 2013: 1665-1672.

[14] Wang W, Lai Q, Fu H, et al. Salient object detection in the deep learning era: An in-depth survey[J]. arXiv preprint arXiv:1904.09146, 2019.

[15] Zhao, Rui, Wanli Ouyang, Hongsheng Li and Xiaogang Wang. Saliency detection by multi-context deep learning. Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition. 2015: 1265-1274.

[16] Li, Guanbin and Yizhou Yu. Visual saliency based on multiscale deep features. Proceedings of the IEEE conference on computer vision and pattern recognition. 2015: 5455-5463.

[17] Kim, Jongpil and Vladimir Pavlovic. A Shape-Based Approach for Salient Object Detection Using Deep Learning. European Conference on Computer Vision. Springer, Cham, 2016: 455-470.

[18] Shelhamer, Evan, Jonathan Long and Trevor Darrell. Fully Convolutional Networks for Semantic Segmentation. IEEE Transactions on Pattern Analysis and Machine Intelligence 39 (2014): 640-651.

[19] Kuen, Jason, Zhenhua Wang and Gang Wang. Recurrent Attentional Networks for Saliency Detection. Proceedings of the IEEE Conference on computer Vision and Pattern Recognition. 2016: 3668-3677.

[20] Wang, Tiantian, Ali Borji, Lihe Zhang, Pingping Zhang and Huchuan Lu. A Stagewise Refinement Model for Detecting Salient Objects in Images. Proceedings of the IEEE International Conference on Computer Vision. 2017: 4019-4028.

[21] Chen, Xiaowu, Anlin Zheng, Jia Li and Feng Lu. Look, Perceive and Segment: Finding the Salient Objects in Images via Two-stream Fixation-Semantic CNNs. Proceedings of the IEEE International Conference on Computer Vision. 2017: 1050-1058.

[22] Hou, Qibin, Ming-Ming Cheng, Xiaowei Hu, Ali Borji, Zhuowen Tu and Philip H. S. Torr. Deeply Supervised Salient Object Detection with Short Connections. Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition. 2017: 3203-3212.

[23] Liu, Nian and Junwei Han. DHSNet: Deep Hierarchical Saliency Network for Salient Object Detection. Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition. 2016: 678-686.

[24] Wu, Zhe, Li Su and Qingming Huang. Cascaded Partial Decoder for Fast and Accurate Salient Object Detection. Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition. 2019: 3907-3916.

[25] Rashwan, Abdullah, Agastya Kalra and Pascal Poupart. Matrix Nets: A New Deep Architecture for Object Detection. Proceedings of the IEEE International Conference on Computer Vision Workshops. 2019: 0-0.

[26] Lin, Tsung-Yi, Piotr Dollár, Ross B. Girshick, Kaiming He, Bharath Hariharan and Serge J. Belongie. Feature Pyramid Networks for Object Detection. Proceedings of the IEEE conference on computer vision and pattern recognition. 2017: 2117-2125.

[27] Yu, Fisher and Vladlen Koltun. Multi-Scale Context Aggregation by Dilated Convolutions. arXiv preprint arXiv:1511.07122, 2015.

[28] Simonyan, Karen and Andrew Zisserman. Very Deep Convolutional Networks for Large-Scale Image Recognition. arXiv preprint arXiv:1409.1556, 2014.

[29] Wang, Lijun, Huchuan Lu, Yifan Wang, Mengyang Feng, Dong Wang, Baocai Yin and Xiang Ruan. Learning to Detect Salient Objects with Image-Level Supervision. Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition. 2017: 136-145.

[30] Yang, Chuan, Lihe Zhang, Huchuan Lu, Xiang Ruan and Ming-Hsuan Yang. Saliency Detection via Graph-Based Manifold Ranking. Proceedings of the IEEE conference on computer vision and pattern recognition. 2013: 3166-3173.

[31] Li, Yin, Xiaodi Hou, Christof Koch, James M. Rehg and Alan L. Yuille. The Secrets of Salient Object Segmentation. Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition. 2014: 280-287.

[32] Shi, Jianping, Qiong Yan, Li Xu and Jiaya Jia. Hierarchical Image Saliency Detection on Extended CSSD. IEEE transactions on pattern analysis and machine intelligence, 2015, 38(4): 717-729.

[33] Li, Guanbin and Yizhou Yu. Visual saliency based on multiscale deep features. Proceedings of the IEEE conference on computer vision and pattern recognition. 2015: 5455-5463.

[34] Ronneberger, Olaf, Philipp Fischer and Thomas Brox. U-Net: Convolutional Networks for Biomedical Image Segmentation. International Conference on Medical image computing and computer-assisted intervention. Springer, Cham, 2015: 234-241.

[35] Xie, Saining and Zhuowen Tu. Holistically-Nested Edge Detection. Proceedings of the IEEE international conference on computer vision. 2015: 1395-1403.

[36] 磐创 News, PytorchChina. PyTorch 官方教程中文版.2020.

[37] Kruthiventi, Srinivas S. S., Vennela Gudisa, Jaley H. Dholakiya and R. Venkatesh Babu. Saliency Unified: A Deep Architecture for simultaneous Eye Fixation Prediction and Salient Object Segmentation. Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition. 2016: 5781-5790.

[38] Fu, Keren, Qijun Zhao, Irene Yu-Hua Gu and Jie Yang. Deepside: A general deep framework for salient object detection. Neurocomputing, 2019, 356: 69-82.

[39] Borji, Ali, Ming Ming Cheng, Huaizu Jiang and Jun-ru Li. Salient Object Detection: A Benchmark. IEEE transactions on image processing, 2015, 24(12): 5706-5722.

[40] 吴恩达,黄海广.Deeplearning.深度学习笔记.2018:380-384.

[41] Perazzi, Federico, Philipp Krähenbühl, Yael Pritch and Alexander Sorkine-Hornung. Saliency filters: Contrast based filtering for salient region detection. 2012 IEEE conference on computer vision and pattern recognition. IEEE, 2012: 733-740.

[42] Fan, Deng-Ping, Ming-Ming Cheng, Yun Liu, Tao Li and Ali Borji. Structure-Measure: A New Way to Evaluate Foreground Maps. Proceedings of the IEEE international conference on computer vision. 2017: 4548-4557.

[43] Liu, Tie, Zejian Yuan, Jian Sun, Jingdong Wang, Nanning Zheng, Xiaoou Tang and Harry Shum. Learning to Detect a Salient Object. IEEE Transactions on Pattern analysis and machine intelligence, 2010, 33(2): 353-367.

[44] Borji A, Cheng M M, Hou Q, et al. Salient object detection: A survey[J]. Computational Visual Media, 2014: 1-34.

[45] Wang L, Wang L, Lu H, et al. Saliency detection with recurrent fully convolutional networks.European conference on computer vision. Springer, Cham, 2016: 825-841.

[46] Cheng, Ming-Ming, Guo-Xin Zhang, Niloy Jyoti Mitra, Xiaolei Huang and Shi-Min Hu. Global contrast based salient region detection. CVPR 2011 (2011): 409-416.